

SAÉ 5.0I : Conception, Réalisation et Présentation d'une solution technique

BUT Réseaux & Télécommunications – 3A (S5)

MediaDock : Serveur multimédia auto-hébergé



Auteurs : Lois GENAY – Vladimir MELTSOV – Kinan HALABI

Encadrant : Michael David

Date : 16/01/2026

Établissement : IUT Nancy-Brabois – Université de Lorraine



1. Contexte et enjeux	2
1.1 Contexte pédagogique.....	2
1.2 Besoin initial.....	2
1.3 Enjeux (R&T).....	2
1.4 Périmètre fonctionnel retenu	3
2. Solution développée et tests	4
2.1 Architecture générale	4
2.2 Backend (Node.js / Express).....	4
2.2.1 Indexation et arborescence.....	5
2.2.2 Streaming vidéo HTTP Range	5
2.2.3 Gestion des chemins et sécurité.....	5
2.3 Frontend (React / Vite).....	6
2.4 Déploiement Docker + CI	8
2.5 Tests et validation	9
2.5.1 Tests fonctionnels.....	9
2.5.2 Tests performances.....	9
3. Journal de bord et adéquation au planning.....	11
3.1 Planning prévisionnel	11
3.2 Réalisé.....	12
3.3 Écarts et justification	13
4. Perspectives et améliorations.....	15
Conclusion	16
Glossaire	17
Bibliographie.....	19

I. Contexte et enjeux

I.1 Contexte pédagogique

Dans le cadre de la SAÉ 5.01, l'objectif est de concevoir, réaliser et présenter une solution technique mobilisant plusieurs compétences du BUT R&T.

Notre projet "MediaDock" s'inscrit dans ce cadre en proposant une application web auto-hébergée permettant d'accéder à une bibliothèque de médias via un navigateur.



I.2 Besoin initial

De nombreux utilisateurs stockent leurs contenus (vidéos, photos, documents) sur un PC, un disque externe ou un NAS. Les solutions existantes (Plex/Jellyfin) peuvent être riches mais parfois lourdes à installer et à maintenir. L'idée de MediaDock est de proposer une alternative **simple** :

- accès via navigateur, sans client dédié
- scan de dossiers montés sur la machine hôte
- navigation par sous-dossiers
- lecture des vidéos en streaming
- galerie photos + prise en charge de **PDF** dans la bibliothèque "Photos"

I.3 Enjeux (R&T)

- **Informatique** : conception d'une API, gestion d'authentification, parcours du système de fichiers, UI web.
- **Réseaux / Télécoms** : diffusion de médias via HTTP avec gestion des requêtes **Range** (lecture progressive), déploiement conteneurisé Docker (ports, volumes, isolation).

- **Déploiement** : reproductibilité (Docker), publication d'image (GHCR) et exécution sur un OS de type NAS (ZimaOS/CasaOS).

I.4 Périmètre fonctionnel retenu

MediaDock couvre :

- Authentification (inscription/connexion/déconnexion)
- Bibliothèque **Vidéos** : navigation dossiers, lecture vidéo via endpoint streaming
- Bibliothèque **Photos** : navigation dossiers, affichage images, ouverture **PDF** dans une modale (iframe)
- Scan manuel ("Rescan")
- Déploiement via Docker (un seul port)

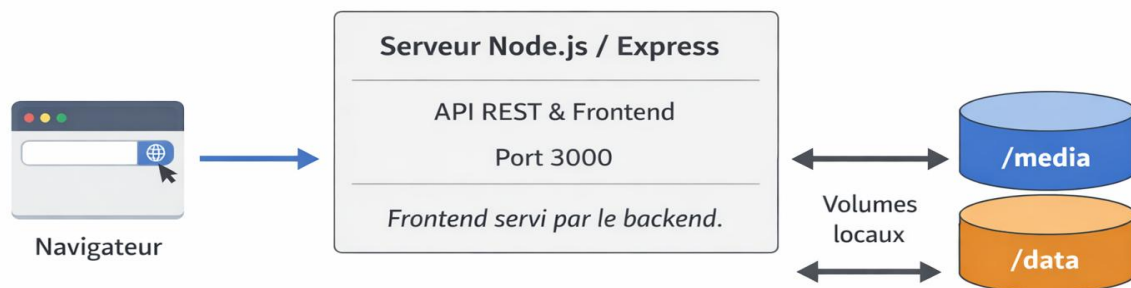
Hors périmètre : métadonnées externes (TMDB/MusicBrainz), gestion musique, transcodage.

2.1 Architecture générale

L'application repose sur un serveur unique Node/Express qui :

1. expose une API REST (auth + bibliothèque)
2. sert les fichiers (stream vidéo / images / PDF)
3. sert le frontend en production (build React)

Schéma :



- `/media` : dossiers vidéos/photos **read**(lecture)
- `/data` : fichier(s) JSON users/sessions **write**(écriture)

2.2 Backend (Node.js / Express)

Variables d'environnement

- `PORT` (défaut 3000)
- `MEDIA_VIDEOS=/media/videos`
- `MEDIA_PHOTOS=/media/photos`
- `DATA_DIR=/data`

Endpoints principaux

- `POST /api/auth/register` : création compte
- `POST /api/auth/login` : connexion (cookie)
- `GET /api/auth/me` : état de session
- `POST /api/auth/logout` : déconnexion
- `GET /api/library` : contenu indexé (listes + arbres)
- `POST /api/scan` : relance scan

Serving médias (protégé par auth)

- `GET /stream?path=...` : vidéo avec support Range
- `GET /image?path=...` : images

2.2.1 Indexation et arborescence

Le scan parcourt récursivement les répertoires montés et produit :

- une liste de fichiers (avec name, path relatif, size, mtime, extension)
 - une arborescence “Tree” pour naviguer par dossier
- Ce choix permet d’afficher rapidement l’arbre à gauche et le contenu du dossier à droite.

2.2.2 Streaming vidéo HTTP Range

Pour permettre la lecture progressive, l’endpoint /stream gère l’en-tête HTTP Range.

- Si Range est présent, le serveur renvoie un 206 Partial Content avec Content-Range et le segment demandé.
 - Sinon, il renvoie le flux complet.
- Ce mécanisme est indispensable pour que le lecteur HTML5 puisse “seek” dans la vidéo et démarrer rapidement.

2.2.3 Gestion des chemins et sécurité

Le backend valide que path reste dans les dossiers autorisés (évite traversée de répertoires).

L’accès aux endpoints médias est conditionné à une session utilisateur valide.

2. Solution développée et tests



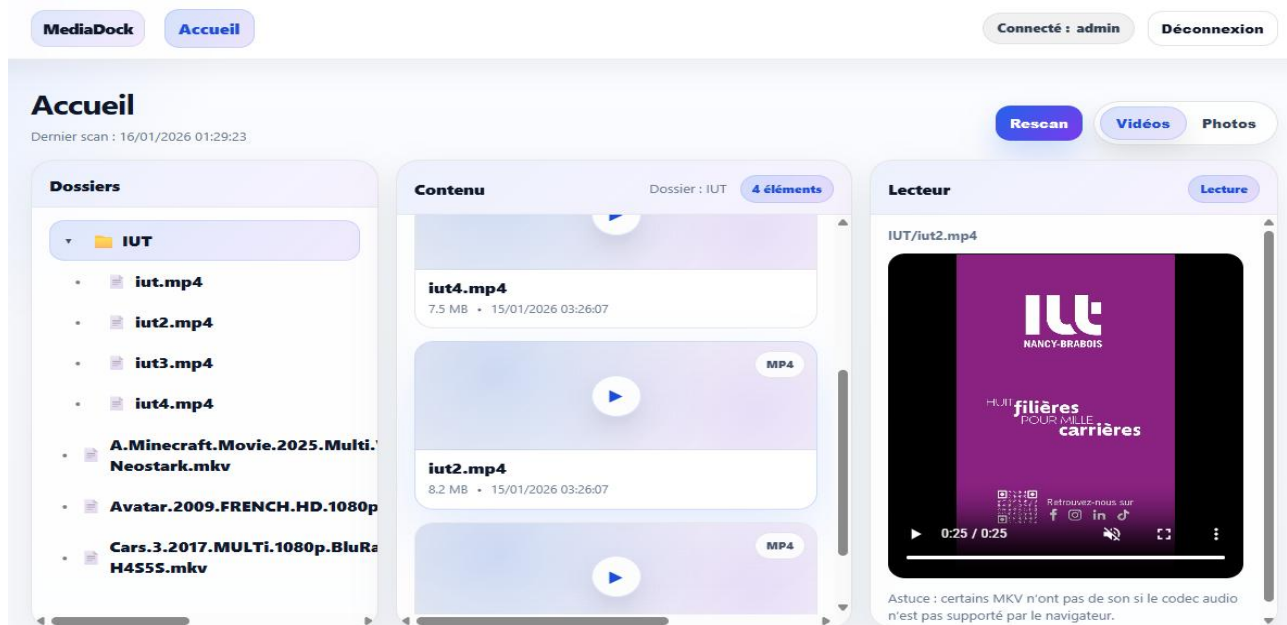
2.3 Frontend (React / Vite)

Pages

Form for user connection. It includes a title 'Connexion', an 'Email' input field, a 'Mot de passe' (password) input field, and a 'Se connecter' button.

Form for user registration. It includes a title 'Inscription', an optional 'Nom (optionnel)' input field, an 'Email' input field, a 'Mot de passe' (password) input field, and a 'Créer le compte' button.

- Connexion / Inscription

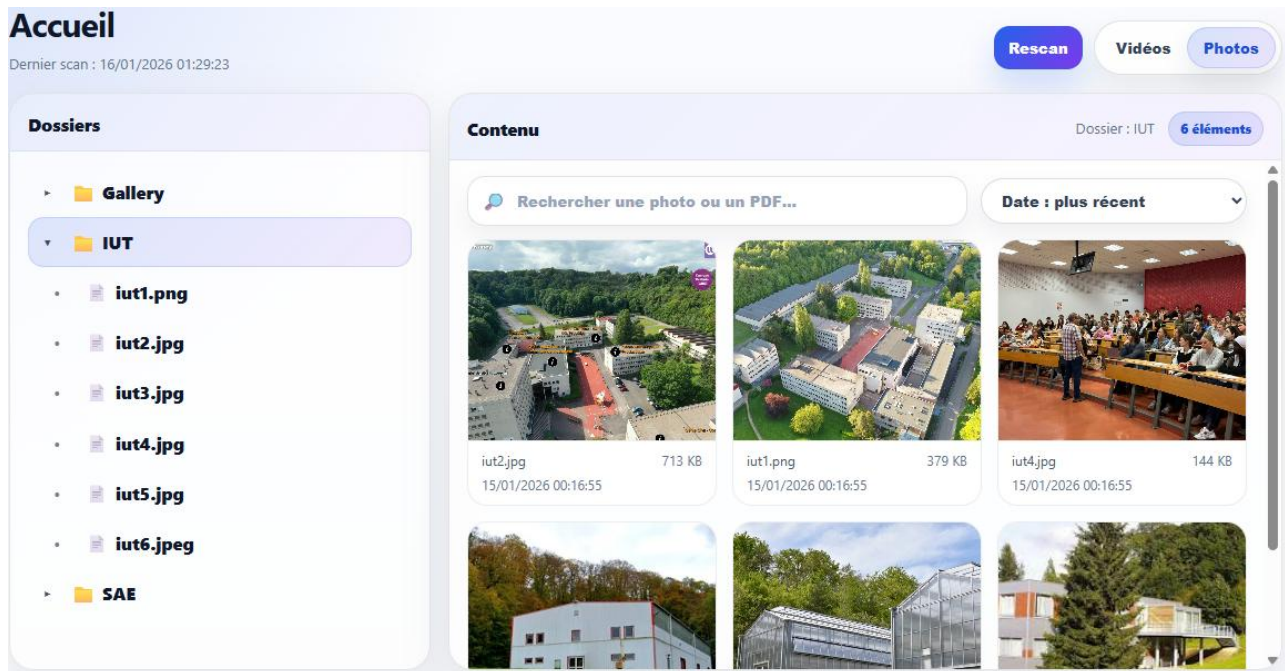


- Accueil : tabs "Vidéos" et "Photos"

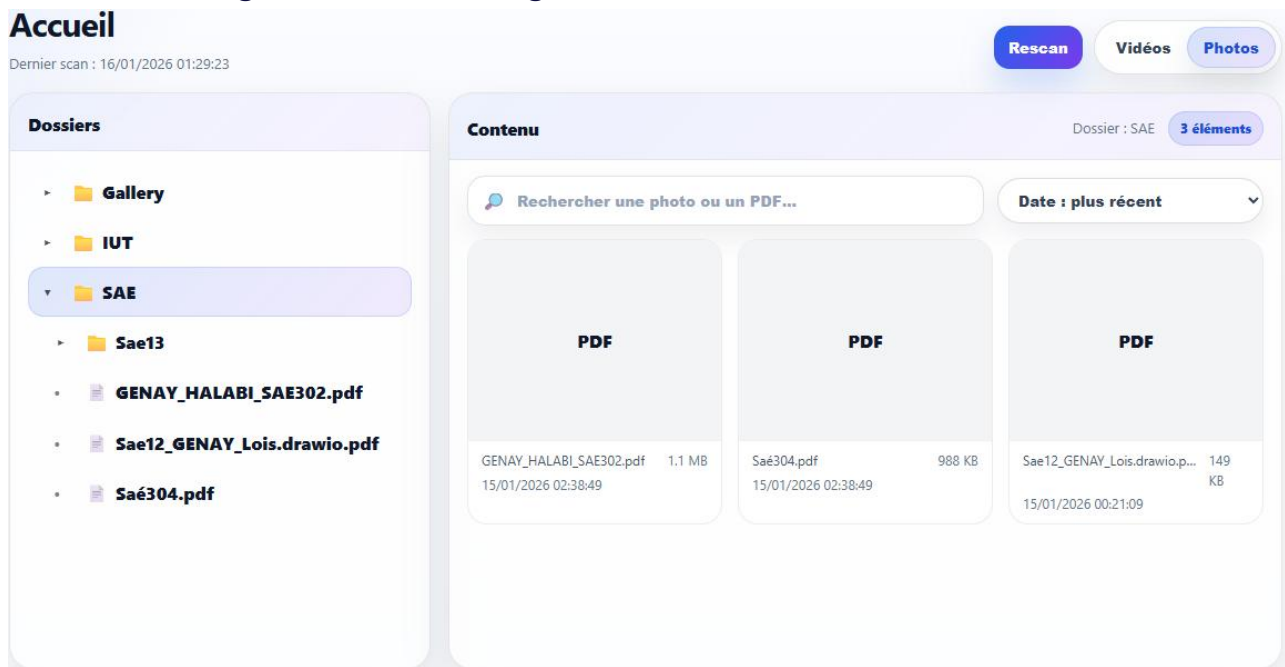
Composants clés

- TreeView : navigation par dossiers (gauche)
- Panneau contenu : cartes fichiers (droite)
- Lecteur vidéo : `<video src="/stream?...">`

2. Solution développée et tests



- Photos : grille + modale image



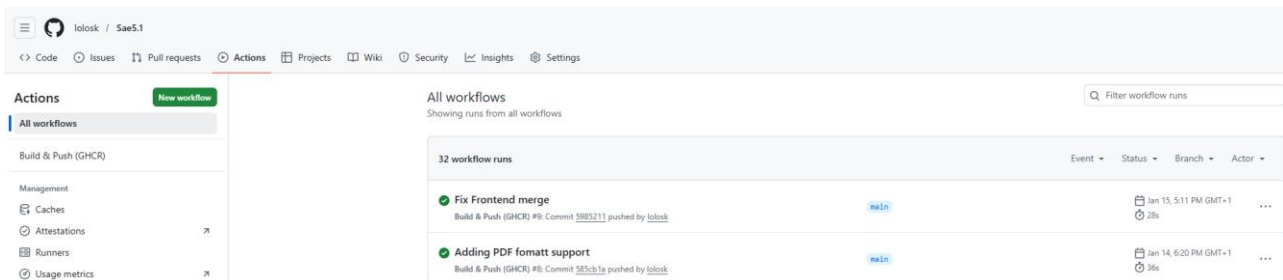
- PDF : carte "PDF" + modale iframe vers /doc?... (ne passe jamais par /image)

Dev vs Prod

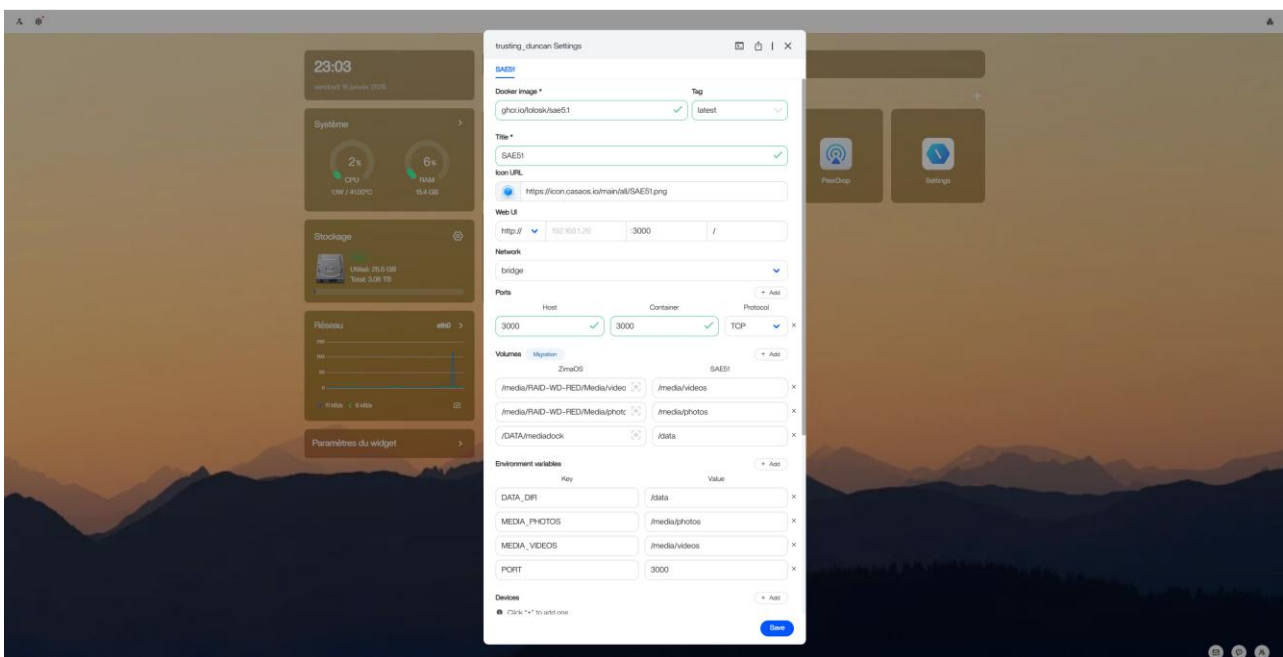
- En développement : Vite + proxy vers l'API
- En production : même origin (port 3000), simplifie cookies et CORS

2. Solution développée et tests

2.4 Déploiement Docker + CI



- Image Docker construite automatiquement et poussée sur GHCR (nom de repo en lowercase).



- Déploiement sur ZimaOS:
 - port 3000:3000
 - volume /media monté sur les médias
 - volume /data pour persister utilisateurs/sessions

2. Solution développée et tests

2.5 Tests et validation

2.5.1 Tests fonctionnels (table)

ID	Fonction	Procédure	Résultat attendu	Statut
T1	Inscription	Créer un compte	Compte créé, connexion possible	OK
T2	Auth requise	Ouvrir / sans login	Redirection/401	OK
T3	Scan	“Rescan”	Arbre + fichiers visibles	OK
T4	Vidéo Range	Lire + avancer	Lecture fluide + seek OK	OK
T5	Image	Ouvrir une photo	Image affichée	OK
T6	PDF	Ouvrir un PDF	Iframe PDF OK	OK
T7	Docker	Lancer conteneur	App accessible + volumes OK	OK

2.5.2 Tests de performance

Les performances ont été évaluées dans un contexte représentatif : application exécutée en **conteneur Docker sur ZimaOS**, accès via **navigateur**. La bibliothèque de test contient environ **22 fichiers** (photos + PDF + vidéos) stockés sur un volume monté **/media**.

Le scan correspond à la génération de l’index (listes + arborescences)

Nom	En-têtes	Aperçu	Réponse	Initiateur	Délai	Cookies
192.168.1.26						
index-D5nxdGX1.js						
index-DielHB6f.css						
me						
favicon.ico						
library						
scan						

En file d'attente depuis 1,19 s		
Début : 1,19 s		
Planification des ressources		DURÉE
Mise en file d'attente ☐		0,65 ms
Début de connexion		DURÉE
Bloqué		0,49 ms

Rescan.

- Nombre total de fichiers scannés : ~22
- Durée du scan initial : ~2,1 s

Observation : la durée dépend principalement du nombre de fichiers et des performances du stockage. Une fois l’index généré, la navigation dans l’UI est rapide car l’arborescence et les listes sont déjà disponibles côté application.

2. Solution développée et tests

Temps de réponse de l'API /api/library

Mesure effectuée sur plusieurs requêtes successives (cache chaud).

- Temps moyen /api/library : ~35 ms
- P95 (optionnel) : ~70 ms

Observation : l'endpoint renvoie une structure JSON (liste + arbre). Le temps varie avec la taille de la bibliothèque (réponse JSON plus lourde lorsque le nombre de fichiers augmente).

Lecture vidéo 1080p sur LAN

Lecture via **lecteur HTML5**, avec démarrage et déplacements dans la timeline.

- Résultat : lecture fluide sur LAN, démarrage rapide et seek fonctionnel grâce au support HTTP Range (206 Partial Content).

Différences entre tests en local (LAN) et accès depuis un réseau extérieur

Les résultats diffèrent fortement selon que l'on accède à MediaDock :

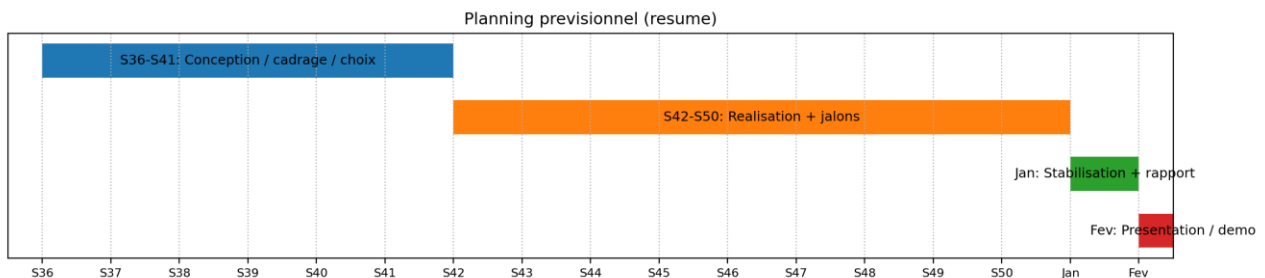
- en local (LAN) : débit élevé et latence faible → le streaming 4k est stable, les seek sont rapides.
- depuis un réseau extérieur (ex. réseau de l'IUT) : le flux dépend de la bande passante disponible, de la latence, et surtout du débit montant du serveur (upload). Même si l'application fonctionne, l'expérience peut être moins stable (mise en mémoire tampon plus fréquente).

Ces écarts sont encore plus visibles sur des contenus 4K, car ils exigent un débit nettement supérieur au 1080p. Sans transcodage adaptatif (hors périmètre du projet), la lecture 4K à distance est plus sensible aux variations de réseau : elle peut rester fluide si le débit est suffisant, mais devient rapidement instable si la bande passante est limitée.

Limites : les fichiers **.mkv** peuvent ne pas avoir de son si le codec audio n'est pas supporté par le navigateur (DTS/AC3). Les formats MP4 (H.264 + AAC) sont recommandés.

3.1 Planning prévisionnel (résumé)

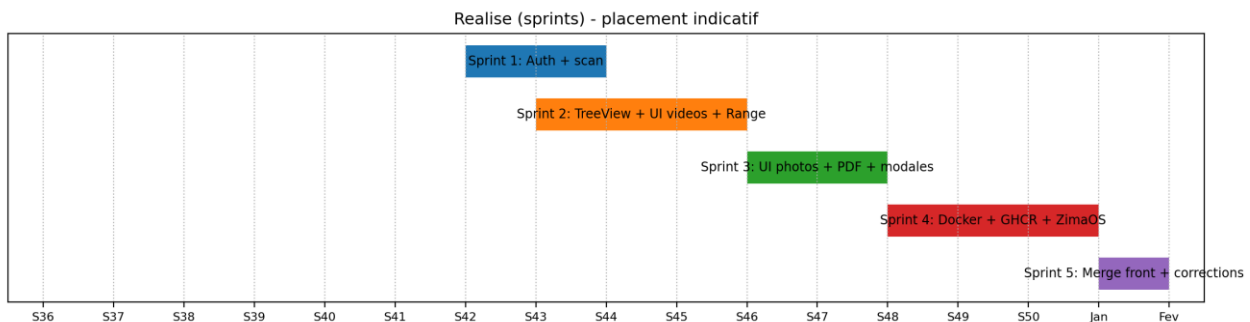
Le projet a quatre grandes phases, alignées sur le calendrier de la SAÉ :



- **S36 à S41** — Conception : cadrage du besoin, définition du périmètre fonctionnel, choix techniques (architecture “serveur unique”, stack Node/Express + React), premières maquettes et stratégie de déploiement.
- **S42 à S50** — Réalisation + jalons intermédiaires : développement backend/frontend, intégration progressive des fonctionnalités (scan, navigation, lecture médias), puis stabilisation technique et tests.
- **Janvier** — Stabilisation + rapport final : corrections, consolidation Docker/GHCR/ZimaOS, rédaction du rapport et préparation des captures/annexes.
- **Février** — Présentation / démonstration à la JPO (Journée Portes Ouvertes) .

3.2 Réalisé (sprints)

La réalisation s'est organisée en sprints fonctionnels, afin de livrer rapidement une base utilisable puis d'ajouter des briques successives :



- Sprint 1 — **Socle backend** (Loïs) : routes d'authentification (register/login/logout/me), stockage local (JSON), endpoint de scan et structure de bibliothèque (listes + arborescence).
- Sprint 2 — **Navigation + Vidéos** (Loïs et Vladimir) : TreeView, UI vidéos (recherche/tri), lecteur HTML5 et streaming via endpoint /stream avec support HTTP Range.
- Sprint 3 — **Photos + PDF** (Loïs et Vladimir) : UI photos, distinction images/PDF, modales (PhotoModal/PdfModal), service PDF via /doc (et non /image).
- Sprint 4 — **Déploiement** (Loïs et Kinan) : conteneurisation Docker, publication GHCR, configuration des volumes /media + /data et déploiement ZimaOS/CasaOS (port 3000).
- Sprint 5 — **Merge frontend + stabilisation** (Loïs, Kinan, Vladimir) : intégration du frontend du binôme, corrections JSX/structures UI, normalisation des comportements (proxy dev / same-origin prod), corrections finales.

3.3 Écarts et justification

Retards / écarts rencontrés

- Intégration frontend ↔ backend : ajustements nécessaires entre les structures JSON renvoyées (library/tree) et l’affichage côté React (tri, filtres, navigation dossiers).
- CORS / proxy en développement : en dev (Vite), certains appels nécessitaient une configuration proxy correcte pour éviter les erreurs réseau ; en production, le serveur unique simplifie car tout est sur le même origin.
- Bug PDF : certains PDF étaient initialement traités comme des images (requêtes envoyées vers /image), entraînant un rendu incorrect.
- Merge du frontend du binôme : conflits et incohérences de structure JSX, nécessitant un nettoyage et une refactorisation de la page Home (balises JSX mal fermées, doublons de grilles, modales).

Fonctionnalités prévues non finalisées (arbitrage)

Certaines fonctionnalités envisagées n’ont pas pu être intégrées dans les délais, principalement en raison du temps consacré à la stabilisation et à l’intégration :

- Mise en place d’indexeurs “avancés” (métadonnées et enrichissement) : l’idée initiale était d’ajouter un système d’indexation plus riche (ex. génération automatique de miniatures, extraction de métadonnées, éventuellement récupération d’informations externes selon les médias). Cette partie a été repoussée car elle demandait davantage de traitement (CPU/stockage), des dépendances supplémentaires et surtout une phase de validation plus longue.
- Reprise de lecture vidéo (resume playback) : la fonctionnalité consistant à reprendre une vidéo à l’endroit exact où l’utilisateur s’est arrêté n’a pas été implémentée. Elle nécessite de stocker une progression par utilisateur et par fichier (ex. timestamp), de la synchroniser avec l’UI (événements du lecteur

HTML5) et de concevoir une structure de persistance fiable (fichier JSON ou base).

Ces choix relèvent d'un arbitrage : priorité donnée à la robustesse du socle (auth, scan, navigation, streaming Range, PDF, Docker + déploiement) plutôt qu'à des fonctionnalités "confort" qui auraient exigé plus de temps de conception et de tests.

Actions correctives mises en place

- Normalisation des réponses API : harmonisation des champs utilisés par le front (path, name, meta, kind, ext) et nettoyage des chemins relatifs.
- Tests via DevTools (Network) : vérification systématique des routes critiques :
 - /stream → 206 Partial Content + Content-Range (Range OK)
 - /doc → Content-Type: application/pdf (PDF OK)
- Correction des modales : séparation stricte des flux :
 - images → /image + PhotoModal
 - PDF → /doc + PdfModal (iframe)
- Stabilisation "serveur unique" en production : un seul port 3000, front servi par le backend, réduction des problèmes de cookies/CORS.

4. Perspectives et améliorations

Même si MediaDock répond au besoin principal (authentification, navigation par dossiers, streaming vidéo, galerie photos et prise en charge des PDF, déploiement Docker), plusieurs améliorations restent envisageables pour aller plus loin, en priorité sur des fonctionnalités à forte valeur ajoutée.

Une première évolution importante serait la **reprise de lecture vidéo** (“resume playback”). L’objectif est de permettre à l’utilisateur de reprendre une vidéo à l’endroit exact où il s’était arrêté. Techniquement, cela consiste à capter côté frontend la position du lecteur HTML5 (événements `timeupdate`, `pause`, `ended`) et à enregistrer régulièrement un timestamp, puis à stocker cette progression par utilisateur et par fichier dans un espace persistant (par exemple dans `/data` sous forme JSON). Au prochain lancement de la vidéo, l’interface pourrait proposer “Reprendre” ou “Recommencer”, et repositionner automatiquement la lecture. Cette fonctionnalité est particulièrement utile en accès distant ou pour des contenus longs.

La deuxième amélioration concerne la mise en place d’**indexeurs** avancés. Aujourd’hui, MediaDock indexe les fichiers à partir du système de fichiers (nom, chemin, taille, date) et construit une arborescence. Un indexeur permettrait d’enrichir la bibliothèque en générant des informations et ressources supplémentaires : miniatures (thumbnails) pour les vidéos et les PDF (ex. capture d’une frame vidéo, première page PDF), extraction de métadonnées utiles (durée, résolution), ou encore optimisation de l’index (cache par dossier, détection de changements) pour réduire le coût d’un rescan. Ces traitements demandent cependant plus de ressources CPU/IO et nécessitent une stratégie de génération (tâche différée, limitation du volume de miniatures, stockage dédié).

Enfin, une perspective plus long terme concerne la compatibilité et la lecture à distance, notamment pour des contenus en 4K. Sans transcodage adaptatif, le streaming dépend fortement de la bande passante et des codecs supportés côté navigateur : en LAN, la lecture est généralement stable, mais sur un réseau extérieur les variations de débit peuvent provoquer des mises en mémoire tampon. Une évolution représente une complexité et une charge machine plus importante.

Le projet **MediaDock** a permis de concevoir et réaliser une solution de serveur multimédia auto-hébergé, simple à déployer et à utiliser via un navigateur.

L'application propose une authentification locale, l'indexation d'une bibliothèque à partir de dossiers montés, la navigation par arborescence, la lecture de vidéos en streaming avec support HTTP Range, ainsi qu'une bibliothèque photos incluant l'ouverture de PDF. L'ensemble est packagé dans une architecture "serveur unique" où le backend Node/Express sert également le frontend en production, ce qui simplifie l'exploitation (un seul port 3000) et rend le déploiement reproductible via Docker.

Au-delà des fonctionnalités, ce travail a mobilisé des compétences centrales du BUT R&T : développement d'API et d'interface web, gestion des accès et du stockage, diffusion de contenus multimédias sur HTTP, et mise en production conteneurisée (volumes, variables d'environnement, publication d'image). Les tests réalisés ont validé le bon fonctionnement des parcours utilisateurs (connexion, scan, navigation, lecture) ainsi que les points techniques essentiels (lecture progressive via 206 Partial Content, service PDF via un endpoint dédié).

Enfin, certaines améliorations restent envisageables pour aller plus loin, notamment la reprise de lecture vidéo, la mise en place d'indexeurs avancés (miniatures/métadonnées) et une meilleure prise en charge de la lecture à distance pour des contenus exigeants (ex. 4K). Malgré ces perspectives, MediaDock constitue une base stable et cohérente, répondant aux objectifs de la SAÉ et démontrant une démarche complète de conception, réalisation, intégration et déploiement.

Terme	Définition
API (Application Programming Interface)	Interface permettant au frontend de communiquer avec le backend via des requêtes (ex. HTTP).
Backend	Partie serveur de l'application. Dans MediaDock : Node.js/Express (auth, scan, distribution des fichiers).
CI/CD	Automatisation build/tests/déploiement. Ici : GitHub Actions pour construire et publier l'image Docker.
CORS	Mécanisme navigateur qui contrôle les requêtes entre origines différentes ; en dev, le proxy Vite évite les blocages.
Docker	Technologie de conteneurisation pour exécuter l'application de façon portable et reproductible.
Endpoint	URL d'un service web correspondant à une fonction (ex. /api/library, /stream, /doc).
Express	Framework Node.js pour créer un serveur web et définir des routes API.
Frontend	Interface utilisateur côté navigateur. Dans MediaDock : React ; en prod, servi par le backend.
GHCR (GitHub Container Registry)	Registre d'images Docker de GitHub utilisé pour stocker/publier l'image MediaDock.
HTTP	Protocole réseau utilisé pour les échanges requête/réponse entre navigateur et serveur.
HTTP Range	Mécanisme HTTP permettant de demander une portion d'un fichier ; utile pour le streaming et le "seek".
206 Partial Content	Code HTTP renvoyé lorsque le serveur répond à une requête Range avec un segment de fichier.

Terme	Définition
Content-Range	En-tête HTTP indiquant la portion de fichier renvoyée (ex. bytes 0-999/12345).
Indexation / Scan	Parcours des répertoires montés pour lister les fichiers et construire l'arborescence.
JSON	Format de données texte utilisé pour échanger les informations entre backend et frontend.
LAN	Réseau local (débit élevé/latence faible), généralement plus stable que l'accès distant.
Node.js	Environnement d'exécution JavaScript côté serveur utilisé pour MediaDock.
Port	Numéro identifiant un service réseau sur une machine. MediaDock utilise le port 3000.
Proxy (Vite)	Redirection en développement pour envoyer /api, /stream, /image, /doc vers le backend local.
Same-origin	Frontend et backend servis depuis la même origine (même IP/domaine + port) ; simplifie cookies et évite le CORS.
Session / Cookie	Mécanisme de maintien de connexion : le serveur gère une session et le navigateur stocke un cookie associé.
Streaming	Lecture progressive d'un média sans téléchargement complet ; dans MediaDock, via HTTP Range.
Volume (Docker)	Montage d'un dossier hôte dans le conteneur pour données persistantes (ex. /media et /data).
ZimaOS	OS orienté NAS permettant de déployer des applications Docker via une interface graphique.

Site de démonstration de l'application

Plateforme multimédia auto-hébergée MediaDock – Projet SAÉ 5.1

Disponible à l'adresse : <http://88.127.188.211:41027/>

Consulté en janvier 2026.

Dépôt GitHub du projet

SAÉ 5.1 – Plateforme multimédia

Dépôt de code source et documentation technique

Disponible à l'adresse : <https://github.com/lolosk/Sae5.1>

Consulté en janvier 2026.